SPRING -2024 BY SADBIN RAHMAN – C241043,WASI-C241041

Group A

1(a): Friend vs. Member Functions

- **Friend Function**: Use it when you need to access private/protected data of two or more classes in a non-member context.
- **Member Function**: Use it when the operation is closely tied to a single class.
- **Example**:

```
class A {
    int x;
    friend void show(const A&);
};
void show(const A& obj) {
    std::cout << obj.x; // Access private member
}
```

1(b): Overload Vector2D Operations

- Define a class Vector2D and overload operators like +, -, and scalar *.

```
void show(const A& obj) {
    std::cout << obj.x; // Access private member
}
class Vector2D {
    float x, y;
public:
    Vector2D(float a = 0, float b = 0) : x(a), y(b) {}
    Vector2D operator+(const Vector2D& v) const { return Vector2D(x +
v.x, y + v.y); }
    Vector2D operator-(const Vector2D& v) const { return Vector2D(x -
v.x, y - v.y); }
    Vector2D operator*(float scalar) const { return Vector2D(x *
scalar, y * scalar); }
};
```

1(B)OR

```
class Time
{
    int hours, minutes, seconds;
public:
    Time(int h = 0, int m = 0, int s = 0) : hours(h), minutes(m),
seconds(s) {}

     Time operator+(const Time &t)
    {
        int total = (hours + t.hours) * 3600 + (minutes + t.minutes) *
60 + seconds + t.seconds;
        return Time(total / 3600, (total % 3600) / 60, total % 60);
    }
    Time operator-(const Time &t) const
    {
        int total = (hours - t.hours) * 3600 + (minutes - t.minutes) *
60 + seconds - t.seconds;
        return Time(total / 3600, (total % 3600) / 60, total % 60);
    }
    Time &operator++()
    {
        ++seconds;
        if (seconds >= 60)
        {
            seconds -= 60;
            ++minutes;
        }
        if (minutes >= 60)
        {
            minutes -= 60;
            ++hours;
        }
        return *this;
    }
```

1(c): Right-Side Argument in Operator Overloading

- In binary operators like + or -, the left operand is the calling object, while the right operand is passed explicitly. This mimics mathematical operations (e.g., a + b).

2(a): Behavior of Public vs. Private Members

- **Public**: Accessible by derived classes and external objects.
- **Private**: Only accessible within the base class.

2(b): Media Class Example

- Example class:

```
#include <iostream>
#include <string>
using namespace std;

// Base class: Media
class Media
{
protected:
    string title;
    string duration;

public:
    // Constructor
    Media(string t, string d) : title(t), duration(d) {}

    // Virtual display function
    virtual void displayInfo() const
    {
        cout << "Title: " << title << ", Duration: " << duration <<
endl;
    }
    // Virtual destructor
    virtual ~Media() {}
};
```

```cpp
// Derived class: Song
class Song : public Media
{
protected:
    string artist;

public:
    // Constructor
    Song(string t, string d, string a) : Media(t, d), artist(a) {}

    // Display function
    void displayInfo() const override
    {
        cout << "Song - Title: " << title << ", Artist: " << artist
             << ", Duration: " << duration << endl;
    }
};

// Derived class: MusicTrack (subclass of Song)
class MusicTrack : public Song
{
private:
    string album;

public:
    // Constructor
    MusicTrack(string t, string d, string a, string al)
        : Song(t, d, a), album(al) {}

    // Display function
    void displayInfo() const override
    {
        cout << "Music Track - Title: " << title << ", Artist: " <<
artist
             << ", Album: " << album << ", Duration: " << duration <<
endl;
    }
};

// Derived class: Podcast
```

```cpp
class Podcast : public Media
{
protected:
    string genre;
    int episodeNumber;

public:
    // Constructor
    Podcast(string t, string d, string g, int ep)
        : Media(t, d), genre(g), episodeNumber(ep) {}

    // Display function
    void displayInfo() const override
    {
        cout << "Podcast - Title: " << title << ", Genre: " << genre
             << ", Episode: " << episodeNumber << ", Duration: " <<
duration << endl;
    }
};

// Derived class: AudioBook (subclass of Podcast)
class AudioBook : public Podcast
{
private:
    string author;

public:
    // Constructor
    AudioBook(string t, string d, string g, int ep, string au)
        : Podcast(t, d, g, ep), author(au) {}

    // Display function
    void displayInfo() const override
    {
        cout << "AudioBook - Title: " << title << ", Genre: " << genre
             << ", Author: " << author << ", Episode: " <<
episodeNumber
             << ", Duration: " << duration << endl;
    }
};
```

```cpp
// Main function to demonstrate functionality
int main()
{
    // Create objects for MusicTrack and AudioBook
    MusicTrack track("Shape of You", "4:24", "Ed Sheeran", "Divide");
    AudioBook book("1984", "8:30:00", "Fiction", 1, "George Orwell");

    // Display information
    Media *media1 = &track;
    Media *media2 = &book;

    cout << "Media Library: " << endl;
    media1->displayInfo();
    media2->displayInfo();
    return 0;
}
```

2(c): Significance of Private Inheritance in C++

- **Private inheritance** means that all public and protected members of the base class become **private** in the derived class.
- This is used when the derived class needs to use the functionality of the base class but does not want to expose it to the outside world, maintaining encapsulation.

```
#include <iostream>
using namespace std;

class Base {
public:
    void display() {
        cout << "Base class function" << endl;
    }
};
class Derived : private Base {
public:
    void show() {
        display(); // Base class's public method is
accessible privately here.
    }
};
int main() {
    Derived d;
    d.show(); // Works.
    // d.display(); // Error: 'display' is private
in Derived.
    return 0;
}
```

2(D): Corrected Code:

```cpp
#include <iostream>
using namespace std;

// Base class: Vehicle
class Vehicle
{
public:
    virtual void start()
    {
        cout << "Vehicle starts" << endl;
    }
    virtual void stop()
    {
        cout << "Vehicle stops" << endl;
    }
};

// Derived class: Car
class Car : public Vehicle
{
public:
    void start() override
    {
        cout << "Car starts" << endl;
    }
    void stop() override
    {
        cout << "Car stops" << endl;
```

```
    }
};

// Derived class: SportsCar
class SportsCar : public Car
{
public:
    void start() override
    {
        cout << "SportsCar starts" << endl;
    }
    void turbo()
    {
        cout << "Turbo Boost!" << endl;
    }
};
```

```
int main()
{
    Vehicle v;
    v.start();
    v.stop();
    Car c;
    c.start();
    c.stop();

    SportsCar s;
    s.start();
    s.turbo();
    s.stop();

    c = s; // Object slicing occurs
    c.start(); // Calls Car's start method
    return 0;
}
```

Corrected Output:

Vehicle starts

Vehicle stops

Car starts

Car stops

SportsCar starts

Turbo Boost!

Car stops

Car starts

3(a):

An **abstract class** is a class that cannot be instantiated directly. It is used to enforce the design of derived classes by providing a common interface. An abstract class contains at least one **pure virtual function**.

Necessity of Abstract Class:

1. **Enforces Consistency**: Ensures derived classes implement specific methods, maintaining a consistent interface.
2. **Prevents Instantiation**: Prevents creation of incomplete objects by disallowing instantiation of the abstract class.
3. **Code Reusability**: Allows common functionality in the base class while requiring derived classes to implement specific behavior.
4. **Polymorphism**: Enables dynamic method binding, allowing different derived classes to be treated uniformly through base class pointers/reference

Example:

```
class Employee {
    virtual void calculateSalary() = 0; // Pure
virtual
};
```

3(b) Design a class hierarchy for a university's personnel management system

Description: **[THIS PART IS ONLY FOR UNDERSTANDING THE CODE]**

1. **Base Class:** Employee
   - Attributes: name, id, salary
   - Functions: calculateSalary() (virtual), displayDetails()
2. **Derived Classes:**
   - Faculty (inherits from Employee)
     - Additional Attributes: subject, researchAllowance
     - Overriden Function: calculateSalary() (to include research allowance)
   - Staff (inherits from Employee)
     - Additional Attributes: department, overtimeHours
     - Overriden Function: calculateSalary() (to include overtime pay)

CODE:

```
#include <iostream>
#include <string>
using namespace std;

// Base Class
class Employee
{
protected:
    string name;
    int id;
    double salary;

public:
    Employee(string n, int i, double s) : name(n), id(i), salary(s) {}
    virtual void calculateSalary()
```

```
    {
        cout << "Base Salary: " << salary << endl;
    }
    virtual void displayDetails()
    {
        cout << "Name: " << name << ", ID: " << id << ", Salary: " <<
salary << endl;
    }
};

// Derived Class: Faculty
class Faculty : public Employee
{
    string subject;
    double researchAllowance;

public:
    Faculty(string n, int i, double s, string subj, double allowance)
        : Employee(n, i, s), subject(subj),
researchAllowance(allowance) {}
    void calculateSalary() override
    {
        salary += researchAllowance;
        cout << "Faculty Salary (including Research Allowance): " <<
salary << endl;
    }
    void displayDetails() override
    {
```

```
        cout << "Name: " << name << ", ID: " << id << ", Subject: " <<
subject
             << ", Salary: " << salary << endl;
    }
};

// Derived Class: Staff
class Staff : public Employee
{
    string department;
    int overtimeHours;

public:
    Staff(string n, int i, double s, string dept, int hours)
        : Employee(n, i, s), department(dept), overtimeHours(hours) {}
    void calculateSalary() override
    {
        salary += overtimeHours * 20; // Overtime pay calculation
        cout << "Staff Salary (including Overtime): " << salary <<
endl;
    }
    void displayDetails() override
    {
        cout << "Name: " << name << ", ID: " << id << ", Department: "
<< department
             << ", Salary: " << salary << endl;
    }
};
```

```
int main()
{
    Faculty f("Alice", 101, 50000, "Mathematics", 5000);
    Staff s("Bob", 102, 30000, "Admin", 10);

    f.calculateSalary();
    f.displayDetails();

    s.calculateSalary();
    s.displayDetails();

    return 0;
}
```

3(B) OR: Simple Banking System

Description: **[THIS PART IS ONLY FOR UNDERSTANDING THE CODE]**

1. **Base Class:** Account
    - Attributes: accountNumber, balance
    - Functions: deposit(), withdraw(), displayBalance()
2. **Derived Classes:**
    - SavingsAccount (inherits from Account)
        - Additional Attributes: interestRate
        - Overriden Function: withdraw() (limit on withdrawal)
    - CheckingAccount (inherits from Account)
        - Additional Attributes: overdraftLimit
        - Overriden Function: withdraw() (overdraft check)

CODE:

```
#include <iostream>
using namespace std;

// Base Class
class Account
{
protected:
    int accountNumber;
    double balance;

public:
    Account(int accNo, double bal) : accountNumber(accNo), balance(bal) {}
    void deposit(double amount)
    {
        balance += amount;
        cout << "Deposited: " << amount << endl;
    }
    virtual void withdraw(double amount)
    {
        balance -= amount;
        cout << "Withdrawn: " << amount << endl;
    }
    void displayBalance()
    {
        cout << "Account Number: " << accountNumber << ", Balance: " << balance << endl;
    }
};

// Derived Class: SavingsAccount
class SavingsAccount : public Account
{
    double interestRate;

public:
    SavingsAccount(int accNo, double bal, double rate)
        : Account(accNo, bal), interestRate(rate) {}
    void withdraw(double amount)
    {
        if (amount > balance)
        {
            cout << "Insufficient balance for withdrawal!" << endl;
        }
        else
        {
```

```
            Account::withdraw(amount);
        }
    }
};

// Derived Class: CheckingAccount
class CheckingAccount : public Account
{
    double overdraftLimit;

public:
    CheckingAccount(int accNo, double bal, double limit)
        : Account(accNo, bal), overdraftLimit(limit) {}
    void withdraw(double amount)
    {
        if (amount > balance + overdraftLimit)
        {
            cout << "Overdraft limit exceeded!" << endl;
        }
        else
        {
            Account::withdraw(amount);
        }
    }
};

int main()
{
    SavingsAccount sa(12345, 1000, 0.03);
    CheckingAccount ca(67890, 2000, 500);

    sa.deposit(500);
    sa.withdraw(200);
    sa.displayBalance();

    ca.deposit(1000);
    ca.withdraw(3000);
    ca.displayBalance();

    return 0;
}
```

3(c)

Drawing a Shape

this is a shape

Drawing Circle

This is Circle

Drawing Rectangle

This is a rectangle

Drawing Circle

this is a shape

Drawing Rectangle

this is a shape

4(a)

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// Generic function to calculate the median
template <typename T>
T CalculateMedian(vector<T> arr)
{
    sort(arr.begin(), arr.end()); // Sort the array
    int n = arr.size();
    return (n % 2 == 0) ? (arr[n / 2 - 1] + arr[n / 2]) / 2.0 : arr[n / 2];
}

int main()
{
    vector<int> intArr = {5, 2, 8, 1, 7};
    vector<double> doubleArr = {3.2, 7.5, 1.8, 9.6};
    vector<char> charArr = {'b', 'z', 'a', 'm', 'h'};

    cout << "Median of integers: " << CalculateMedian(intArr) << endl;
    cout << "Median of doubles: " << CalculateMedian(doubleArr) << endl;
    cout << "Median of characters: " << CalculateMedian(charArr) << endl;

    return 0;
}
```

4(a)or

```cpp
#include <iostream>
#include <vector>
using namespace std;
// Template class for a generic array
template <typename T>
class GenericArray {
    vector<T> elements;
public:
    GenericArray(vector<T> arr) : elements(arr) {}

    T calculateSum() {
        T sum = 0;
        for (T elem : elements) {
            sum += elem;
        }
        return sum;
    }
};

int main() {
    GenericArray<int> intArray({1, 2, 3, 4, 5});
    cout << "Sum of integers: " << intArray.calculateSum() <<
endl;

    GenericArray<double> doubleArray({1.1, 2.2, 3.3, 4.4});
    cout << "Sum of doubles: " << doubleArray.calculateSum() <<
endl;

    GenericArray<char> charArray({'a', 'b', 'c'});
    cout << "Sum of characters (ASCII values): " <<
(int)charArray.calculateSum() << endl;

    return 0;
}
```

4(b)

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> vec = {5, 10, 2, 55, 60, 3};

    // Remove the element at index 2
    vec.erase(vec.begin() + 2);

    // Insert 15 at index 1
    vec.insert(vec.begin() + 1, 15);

    // Remove the first element
    vec.erase(vec.begin());

    // Print the final vector
    cout << "Final vector: ";
    for (int val : vec)
    {
        cout << val << " ";
    }
    cout << endl;

    return 0;
}
```

4(c): Exception Handling

What is an Exception? An exception is an event that occurs during the execution of a program, disrupting the normal flow of the program's instructions.

Advantages of Exception Handling:

1. **Error Separation**: Separates error-handling code from regular code.
2. **Error Propagation**: Allows functions to propagate errors up the call stack.
3. **Code Clarity**: Makes the code more readable and maintainable.
4. **Recovery Mechanism**: Allows the program to recover from runtime errors.

[THIS FOLLOWING CODE IS ONLY FOR UNDERSTANDING E.H.]

```
#include <iostream>
#include <exception>
using namespace std;

int main()
{
    try
    {
        int a = 10, b = 0;
        if (b == 0)
            throw runtime_error("Division by zero error");
        cout << "Result: " << a / b << endl;
    }
    catch (const exception &e)
    {
        cout << "Exception caught: " << e.what() << endl;
    }
    return 0;
}
```

5(A)

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    // I. Hexadecimal with width 10 and fill '0'
    cout << setw(10) << setfill('0') << hex << 255 << endl;

    // II. Octal with width 10 and fill '0'
    cout << setw(10) << setfill('0') << oct << 493 << endl;

    // III. Float with width 10 and precision 2
    cout << setw(10) << fixed << setprecision(2) << 123456.789
<< endl;

    return 0;
}
```

5(a) or

```cpp
#include <iostream>
#include <iomanip> // For manipulators
using namespace std;

int main()
{
    double value = 1234.56789;

    // Custom manipulator to format the float
    cout << setw(12)        // Width of 12
         << right           // Right justified
         << fixed           // Fixed-point notation
         << setprecision(2) // Precision of 2
         << setfill('*')    // Fill with '*'
         << value           // Output the value
         << endl;
```

5(b)

Ios class:

```cpp
#include <iostream>
#include <iomanip> // For manipulators
using namespace std;

int main() {
    int num = 12345;

    // Set width and fill using ios class
    cout.width(10);   // Set field width
    cout.fill('*');   // Set fill character
    cout << num << endl; // Output: *****12345

    // Set precision using ios class
    cout.precision(4);  // Set precision
    cout << fixed << 123.45678 << endl; // Output: 123.4568

    return 0;
}
```

Manupulators:

```cpp
#include <iostream>
#include <iomanip> // For manipulators
using namespace std;

int main() {
    int num = 12345;
    // Use manipulator for setting width and fill
    cout << setw(10) << setfill('*') << num << endl;  //
Output: *****12345

    // Use manipulator for setting precision
```

```
    cout << fixed << setprecision(4) << 123.45678 << endl; // Output: 123.457

    return 0;
}
```

5(c)

```
#include <iostream>
#include <fstream>
#include <vector>
using namespace std;

int main()
{
    // Write 100 integers to RAND.TXT
    ofstream outFile("RAND.TXT");
    for (int i = 1; i <= 100; i++)
    {
        outFile << i << " ";
    }
    outFile.close();

    // Read contents from RAND.TXT
    ifstream inFile("RAND.TXT");
    int number;
    cout << "File contents: ";
    while (inFile >> number)
    {
        cout << number << " ";
    }
    inFile.close();
    return 0;
}
```